



PRACTICAL LAB RECORD

University	Dev Sanskriti Vishwavidyalaya, Haridwar
Course	Bachelor of Computer Applications (BCA)
Subject	CS302JSC - JavaScript
Assignment	Practical Lab Session 5 - Functions & Variable Scope
Submitted By	Rishikesh Priyadarshi

Part 1: Function Declaration Basics

- Task 1.1:** Write a function declaration `isAdult(age)` that returns `true` if `age >= 18`, otherwise `false`. Test it with 3 different ages.
- Task 1.2:** Write a function declaration `calculateDiscount(price, isMember)` that returns `price * 0.9` if `isMember` is `true`, otherwise returns price unchanged.

Source Code:

```
// Task 1.1: isAdult Function Declaration
function isAdult(age) {
    return age >= 18;
}
console.log("Adult check (15):", isAdult(15));
console.log("Adult check (18):", isAdult(18));
console.log("Adult check (22):", isAdult(22));

// Task 1.2: calculateDiscount Function Declaration
function calculateDiscount(price, isMember) {
    if (isMember) {
        return price * 0.9;
    }
    return price;
}
console.log("Member price:", calculateDiscount(100, true));
console.log("Non-member price:", calculateDiscount(100, false));
```

Part 2: Function Expressions & Arrow Functions

1. **Task 2.1:** Rewrite the `isAdult` function as a function expression.
2. **Task 2.2:** Write an arrow function `square(n)` that returns $n * n$, using the shortest arrow form.
3. **Task 2.3:** Write an arrow function `fullName(first, last)` that returns the two names joined with a space.

Source Code:

```
// Task 2.1: isAdult as a Function Expression
const isAdultExpr = function(age) {
  return age >= 18;
};
console.log("Expression adult check (20):", isAdultExpr(20));

// Task 2.2: Quick Square Arrow Function
const square = n => n * n;
console.log("Square of 4:", square(4));
console.log("Square of 7:", square(7));
console.log("Square of 10:", square(10));

// Task 2.3: fullName Arrow Function
const fullName = (first, last) => `${first} ${last}`;
console.log("Full Name:", fullName("Rishikesh", "Priyadarshi"));
```

Part 3: Default Parameters & Argument Mismatch

1. **Task 3.1:** Write a function `calculatePrice(price, tax=0.18)` that returns $\text{price} + (\text{price} * \text{tax})$.
2. **Task 3.2:** Call `calculateArea(length, width)` with only ONE argument. What do you get, and why?

Source Code & Investigation:

```
// Task 3.1: calculatePrice with Default Tax
function calculatePrice(price, tax = 0.18) {
  return price + (price * tax);
}
console.log("Custom tax (5%):", calculatePrice(100, 0.05));
console.log("Default tax (18%):", calculatePrice(100));

// Task 3.2: Argument Mismatch Investigation
function calculateArea(length, width) {
  return length * width;
}
console.log("Area with one argument:", calculateArea(5));
```

Observation for Task 3.2: Calling the function with only one argument logs NaN (Not-a-Number). This occurs because the missing width argument defaults to undefined, and attempting mathematical multiplication on undefined results in NaN.

Part 4: Global vs Local Scope

1. **Task 4.1:** Create a global variable `taxRate = 0.18`. Write `finalPrice(amount)` that uses the global `taxRate`.
2. **Task 4.2:** Inside a function, declare a local variable with the exact SAME NAME as a global variable. Print the value inside and outside.

Source Code & Investigation:

```
// Task 4.1: finalPrice with Global Variable
let taxRate = 0.18;
function finalPrice(amount) {
  return amount + (amount * taxRate);
}
console.log("Final Price:", finalPrice(200));

// Task 4.2: Scope Shadowing Investigation
let storeName = "QuickMart";
function printStore() {
  let storeName = "LocalMart"; // Local declaration shadows global
  console.log("Inside function:", storeName);
}
printStore();
console.log("Outside function:", storeName);
```

Observation for Task 4.2: The global variable remains unchanged as "QuickMart". The local variable declaration creates a separate, temporary variable that "shadows" the global one only during the function's execution.

Part 5: Block Scope: let vs var

1. **Task 5.1:** Write an `if` block declaring a variable with `let` inside it.
2. **Task 5.2:** Repeat Task 5.1, but declare the variable with `var`.

Source Code & Investigation:

```
// Task 5.1: Block Scope with let
if (true) {
  let discountApplied = true;
  console.log("Inside block (let):", discountApplied);
}
// console.log(discountApplied); // Throws ReferenceError if uncommented

// Task 5.2: Block Scope with var
if (true) {
  var discountVar = true;
}
console.log("Outside block (var):", discountVar);
```

Observation for Task 5.2: The `var` declaration ignores the block boundaries of the `if` statement and remains accessible outside of it, demonstrating that `var` is function-scoped rather than block-scoped.

Part 6: Scope Chain & Shadowing

1. **Task 6.1:** Write a nested function where the inner function reads a variable declared in the outer function.
2. **Task 6.2:** Create a global variable `role = "guest"`. Write `loginAsAdmin()` that declares a `LOCAL role = "admin"`.

Source Code:

```
// Task 6.1: Nested Functions
function checkStatus() {
  let status = "Active";
  function displayStatus() {
    console.log("Nested function reading outer var:", status);
  }
  displayStatus();
}
checkStatus();

// Task 6.2: Secret Admin Mode
let role = "guest";
function loginAsAdmin() {
  let role = "admin";
  console.log("Inside loginAsAdmin:", role);
}
loginAsAdmin();
console.log("Outside loginAsAdmin (Global):", role);
```

Part 7: Mini Project: Student Grade & Fee Manager

Requirements: Combine declarations, expressions, and arrow functions to calculate student grades, apply late fees, modify a global total, and print summaries.

Source Code:

```
let totalFeeCollected = 0;

// Function Declaration for grading
function calculateGrade(marks) {
  if (marks >= 90) return "A";
  if (marks >= 75) return "B";
  if (marks >= 60) return "C";
  return "F";
}

// Function Expression for late fees
const calculateLateFee = function(daysLate = 0) {
  return daysLate * 10;
};
```

```
// Arrow Function for main processing
const processStudent = (name, marks, daysLate = 0) => {
  let grade = calculateGrade(marks);
  let fee = calculateLateFee(daysLate);

  totalFeeCollected += fee;

  console.log(`${name} -> Grade ${grade}, Late Fee Rs.${fee}.`);
};

// Execution / Testing
processStudent("Aditi", 92, 0);
processStudent("Rohit", 68, 3);
processStudent("Meera", 55, 5);
console.log(`Final totalFeeCollected -> Rs.${totalFeeCollected}.`);
```

Part 8: Debugging Challenge

Snippet 1 Analysis & Fix

- **Expected Behavior:** Return the arithmetic sum of arguments a and b.
- **Actual Behavior:** Returns undefined.
- **Why it failed:** The operation a + b executes, but the result is not returned to the caller due to the missing return keyword.
- **Fixed Code:**

JavaScript

```
function addNumbers(a, b) {
  return a + b;
}
console.log(addNumbers(5, 3));
```

Snippet 2 Analysis & Fix

- **Expected Behavior:** Assign the value 20 to a variable.
- **Actual Behavior:** Creates an implicit, undeclared global variable (which will trigger a ReferenceError in strict mode).
- **Why it failed:** The let, const, or var keyword was omitted during variable creation.
- **Fixed Code:**

JavaScript

```
function setDiscount() {  
  let discount = 20;  
}
```

Snippet 3 Analysis & Fix

- **Expected Behavior:** Subtract 200 from the global balance of 1000, leaving 800.
- **Actual Behavior:** The global balance remains 1000.
- **Why it failed:** The function parameter is named balance, which locally shadows the global variable of the same name. The subtraction modifies the local parameter, not the global state.
- **Fixed Code:**

```
let balance = 1000;  
function withdraw(amount) {  
  balance = balance - amount;  
  return balance;  
}  
withdraw(200);  
console.log(balance);
```

Snippet 4 Analysis & Fix

- **Expected Behavior:** Print "Hi!" to the console.
- **Actual Behavior:** Throws a ReferenceError indicating sayHello cannot be accessed before initialization.
- **Why it failed:** Function expressions assigned to const variables are subject to the temporal dead zone and are not hoisted in the same manner as standard function declarations. They must be defined before they are called.
- **Fixed Code:**

```
const sayHello = function() {  
  console.log("Hi!");  
};  
sayHello();
```